

EgoModelKit Machine Setup

Purpose

This procedure is the administrator deployment guide for setting up the machine pre-requisites for Windows with WSL 2 and Ubuntu Linux machines.

1. Windows setup

A. Administrator instructions (once per machine)

I. Confirm virtualization and SLAT

Open Task Manager > Performance > CPU and confirm Virtualization is Enabled. Enable Intel VT-x or AMD-V/SVM in BIOS/UEFI when needed. The processor must support SLAT.

II. Verify the NVIDIA driver

```
winver  
nvidia-smi
```

III. Install Google Chrome

Use the official Google Chrome download, install the current stable 64-bit release, open it once, and allow updates.

IV. Install Windows Python

Install official 64-bit Python 3.10 or newer with Tcl/Tk, Python file associations, and Add Python to PATH. Ensure Python installer is used to add Python system-wide system wide with the following dependencies:

```
pip
tcl/tk and IDLE
Python test suite
py launcher
```

Ensure the below configuration is enabled:

```
Install Python for all users
Associate files with Python
Create shortcuts for installed applications
Add Python to environment variables
```

Preferably keep Python installation in:

```
C:\Program Files\Python313
```

Verify Tkinter from PowerShell.

```
python --version
python -c "import tkinter; print(tkinter.TkVersion)"
```

V. Enable and update WSL machine-wide

Sign in to the Windows administrator account and open PowerShell as administrator. Enable WSL without installing a distribution under the administrator account, update WSL, select WSL 2 as the default, and restart Windows when requested.

```
wsl --install --no-distribution
wsl --update
wsl --set-default-version 2
wsl --version
```

VI. Install Docker Desktop

Sign in to the administrator account and install the current Docker Desktop for Windows. Use Linux containers and the WSL 2 backend. Do not install a second Docker Engine inside the Ubuntu WSL distribution.

VII. Enable WSL integration

Open Docker Desktop settings. Under General, enable the WSL 2 engine. Under Resources > WSL Integration, enable Ubuntu-24.04 for the target account. Apply and restart Docker Desktop.

B. Administrator instructions (once per target Windows account)

I. Sign in to the exact target Windows account

Sign out of the administrator account and sign in to the standard Windows account that will run EgoModelKit. The administrator performs the following steps using the approved account-provisioning process; the target user does not need to participate.

II. Open PowerShell normally and confirm the account

Open PowerShell or Windows Terminal normally - do not select Run as administrator. Confirm the command is running as the intended Windows account.

```
whoami
```

III. Install Ubuntu for this Windows account

Check the online distribution name, then install Ubuntu for the currently signed-in target account. The `--web-download` option avoids depending on a Start-menu or Microsoft Store shortcut. Use the exact name shown by `wsl --list --online`. A reboot of the system may be necessary after the following steps:

```
wsl --list --verbose  
wsl --list --online  
wsl --install --web-download -d Ubuntu-24.04
```

IV. Start Ubuntu directly and create the Linux account

When installation finishes, start the distribution from the same normal PowerShell window. At the prompts, create a recognizable Linux username and password on behalf of the target user. No characters appear while the password is typed; this is normal.

```
wsl --distribution Ubuntu-24.04
```

V. Verify the Linux account and WSL version

Inside Ubuntu, confirm the Linux username and exit. Back in PowerShell, confirm the distribution is listed with VERSION 2. Convert it if VERSION 1 is shown.

```
# Inside Ubuntu
whoami
exit
# Back in PowerShell
wsl --list --verbose
# only if VERSION 1 is shown
wsl --set-version Ubuntu-24.04 2
```

VI. Open the target account's Ubuntu distribution

Open PowerShell normally and start that account's Ubuntu distribution directly; a Start-menu shortcut is not required.

```
wsl --distribution Ubuntu-24.04
```

VII. Install Linux-side tools

Inside Ubuntu, use the Linux password created by the administrator to install Git, Python virtual-environment support, and certificate tools.

```
sudo apt update
sudo apt install -y git python3 python3-venv ca-certificates
```

VIII. Validate the target account

Still inside the target account's Ubuntu distribution, run every command below. The final command may download a CUDA image. If nvidia-smi is not on PATH, try `/usr/lib/wsl/lib/nvidia-smi`.

If the docker version command fails, then may have to manually open the Docker Desktop application and ensure another signed-in account is not running Docker Desktop to avoid permission issues. Also may need to rerun the following on the target user account: Open Docker Desktop settings. Under General, enable the WSL 2 engine. Under Resources > WSL Integration, enable Ubuntu-24.04 for the target account. Apply and restart Docker Desktop.

```
python3 --version
git --version
nvidia-smi
docker version
docker run --rm --gpus all nvidia/cuda:12.8.1-base-ubuntu22.04
nvidia-smi
```

C. User instructions

I. Download EgoModelKit launchers

Download the Python EgoModelKit launchers file ([egomodelkit_machine_setup.py](#) and [egomodelkit_machine_setup.bat](#)) and place it in the Downloads folder.

II. Start EgoModelKit launchers

Double-click on the `egomodelkit_machine_setup.bat` file and run `egomodelkit_machine_setup.py`.

III. Follow the graphical setup window

The graphical setup window will allow the user to start, stop, uninstall, and check diagnostic logs associated with EgoModelKit.

2. Linux setup

A. Administrator instructions (once per machine)

I. Update Ubuntu and install the NVIDIA driver

Install approved updates and ubuntu-drivers-common. Use Additional Drivers or ubuntu-drivers, restart, and confirm `nvidia-smi` works.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ubuntu-drivers-common
sudo ubuntu-drivers install
sudo reboot
nvidia-smi
```

II. Remove conflicting Docker packages

Review and remove conflicting distribution or third-party Docker packages when installed.

```
sudo apt remove -y docker.io docker-doc docker-compose \
docker-compose-v2 podman-docker containerd runc
```

III. Add Docker's Ubuntu repository

```
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
CODENAME="$(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")"
ARCH="$(dpkg --print-architecture)"
```

```
printf 'Types: deb\nURIs: https://download.docker.com/linux/ubuntu\n'
\
'Suites: %s\nComponents: stable\nArchitectures: %s\n' "$CODENAME"
"$ARCH" | \
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null
printf 'Signed-By: /etc/apt/keyrings/docker.asc\n' | \
sudo tee -a /etc/apt/sources.list.d/docker.sources >/dev/null
```

IV. Install and start Docker Engine

Install Docker Engine and its standard build and Compose plugins, then enable the service.

```
sudo apt update
```

If the above command fails due to the Docker repository file being malformed, then run these commands from your sudo-capable Ubuntu account to replace it with Docker's official deb822-format repository file.

```
sudo rm -f /etc/apt/sources.list.d/docker.sources
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL \
    https://download.docker.com/linux/ubuntu/gpg \
    -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Load the Ubuntu release information:

```
. /etc/os-release
```

Now create the corrected source file:

```
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: ${UBUNTU_CODENAME:-$VERSION_CODENAME}
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Inspect it:

```
cat /etc/apt/sources.list.d/docker.sources
```

On Ubuntu 24.04 x86-64, it should resemble:

```
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
```

Then retry:

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

V. Install NVIDIA Container Toolkit

Add NVIDIA's repository, install the toolkit, configure Docker, and restart Docker.

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
sudo gpg --dearmor -o
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/\
nvidia-container-toolkit.list | sed \
's#deb https://#deb
[signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | \
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt update && sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```


VI. Install system-wide user tools and Chrome

Install Git, Python, venv, Tkinter, certificates, and desktop helpers. Install the current stable 64-bit Google Chrome from the official Debian/Ubuntu package and open it once.

```
sudo apt install -y git python3 python3-venv python3-tk
ca-certificates xdg-utils
cd ~/Downloads
curl -fL \
-o google-chrome-stable_current_amd64.deb \
https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.
deb
sudo apt install -y ./google-chrome-stable_current_amd64.deb
google-chrome --version
```

VII. Add every intended user to the docker group

Replace USERNAME with the Linux username of each non-technical lab user or other account that will run EgoModelKit. Run the command once per target account.

```
sudo usermod -aG docker USERNAME
```

VIII. Refresh group membership

For each target account, sign out completely and sign back in, or restart the machine. Merely closing a terminal is not sufficient.

B. Administrator instructions (once per target Linux account)

I. Validate as the target user without sudo

Sign in to each target account and run every command below. Do not use sudo. The administrator performs this validation before handoff.

```
python3 --version
python3 -c "import tkinter; print(tkinter.TkVersion)"
git --version
google-chrome --version
nvidia-smi
docker version
docker run --rm --gpus all nvidia/cuda:12.8.1-base-ubuntu22.04
nvidia-smi
```

C. User instructions

I. Download EgoModelKit launcher

Download the Python EgoModelKit launcher file ([egomodelkit_machine_setup.py](#)) and place it somewhere on the machine, such as the Downloads folder.

II. Make the EgoModelKit launcher an executable program

Right-click egomodelkit_machine_setup.py > Properties > Permissions and allow execution as a program.

III. Start EgoModelKit launcher

Right-click egomodelkit_machine_setup.py and choose Run as a Program.

IV. Follow the graphical setup window

The graphical setup window will allow the user to start, stop, uninstall, and check diagnostic logs associated with EgoModelKit.